CrewAI: Multi-Agent AI Framework

Complete Notes with Code Examples and a Mini Project

---

1. Introduction to CrewAI

What is CrewAI?

CrewAI is an open-source Python framework used to build **multi-agent AI systems** where multiple AI agents collaborate to solve complex problems.

Unlike a single chatbot, CrewAI allows multiple specialized AI agents to work together, each with its own role, goals, and tools.

For example, in a company:

- CEO plans strategy.
- Research Team gathers information.
- Marketing Team creates campaigns.
- Sales Team contacts customers.

Similarly, CrewAI creates AI agents with different responsibilities.

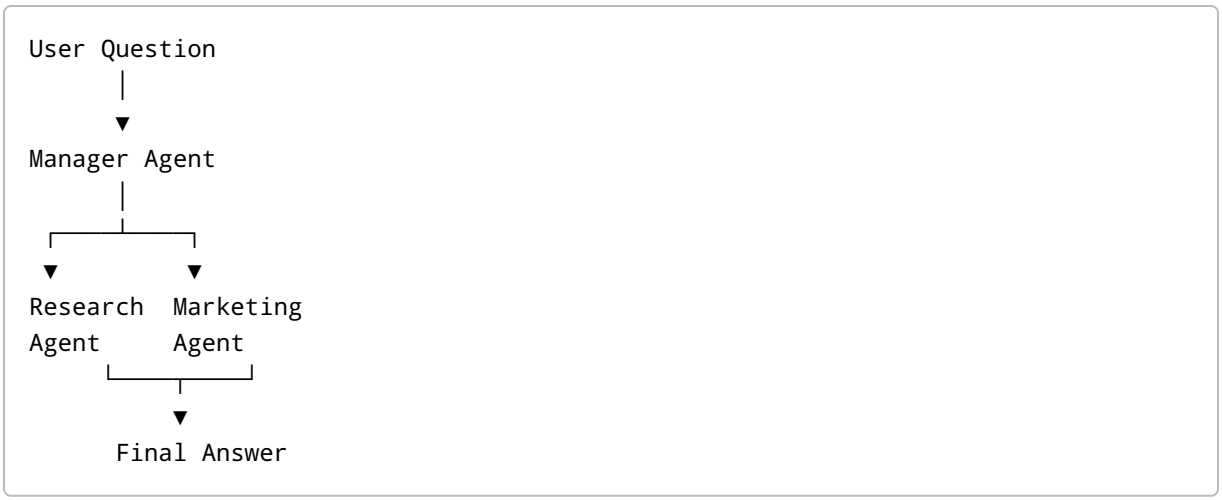
Why Use CrewAI?

Traditional AI applications use a single Large Language Model (LLM).

Example:

```
User Question
      │
      ▼
Single LLM
      │
      ▼
Answer
```

CrewAI uses multiple specialized agents.



Features

- Multi-Agent Collaboration
- Role-Based Agents
- Task Delegation
- Tool Integration
- Memory Support
- Sequential Workflow
- Hierarchical Workflow

Installation

```
pip install crewai
pip install crewai-tools
pip install langchain-google-genai
```

Project Structure

```
crewai_project/

├── app.py
├── agents.py
├── tasks.py
├── crew.py
```

```
├── tools.py
├── requirements.txt
```

---

2. Core Concepts

CrewAI consists of four main components:

1. Agent
2. Task
3. Tool
4. Crew

Agent

An Agent is an AI worker.

Example:

```python
from crewai import Agent

researcher = Agent(
    role="Research Analyst",
    goal="Find accurate information",
    backstory="Expert in market research",
    verbose=True
)
```

Parameters

Parameter	Description
role	Agent's designation
goal	Primary objective
backstory	Background knowledge
verbose	Show execution logs
tools	Tools available to the agent
llm	Language model

Task

A Task tells an agent what to do.

```
from crewai import Task

task = Task(
    description="Research Artificial Intelligence",
    expected_output="Detailed summary",
    agent=researcher
)
```

Crew

A Crew combines multiple agents.

```
from crewai import Crew

crew = Crew(
    agents=[researcher],
    tasks=[task]
)

result = crew.kickoff()

print(result)
```

Tool

Tools allow agents to interact with external systems.

Examples:

- Web Search
- Calculator
- File Reader
- SQL Database
- APIs

3. Creating Multiple Agents

```
from crewai import Agent

research_agent = Agent(
    role="Research Analyst",
    goal="Research companies",
    backstory="Expert researcher",
    verbose=True
)

marketing_agent = Agent(
    role="Marketing Expert",
    goal="Create marketing strategies",
    backstory="Digital marketing specialist",
    verbose=True
)

sales_agent = Agent(
    role="Sales Executive",
    goal="Write personalized sales emails",
    backstory="Experienced B2B salesperson",
    verbose=True
)
```

4. Creating Tasks

```
from crewai import Task

research_task = Task(
    description="Research OpenAI company",
    expected_output="Company profile",
    agent=research_agent
)

marketing_task = Task(
    description="Create marketing strategy",
    expected_output="Marketing plan",
    agent=marketing_agent
)
```

```
sales_task = Task(
    description="Write cold email",
    expected_output="Professional email",
    agent=sales_agent
)
```

---

5. Creating a Crew

```
from crewai import Crew

crew = Crew(
    agents=[
        research_agent,
        marketing_agent,
        sales_agent
    ],
    tasks=[
        research_task,
        marketing_task,
        sales_task
    ],
    verbose=True
)

result = crew.kickoff()

print(result)
```

---

6. Using Tools

Example: Calculator Tool

```
from crewai.tools import tool

@tool("Calculator")
def calculator(expression: str):
    """
    Evaluate a mathematical expression.
```

```
    """
    return str(eval(expression))
```

Agent:

```
agent = Agent(
    role="Math Expert",
    goal="Solve mathematical problems",
    tools=[calculator]
)
```

Example Tool: Current Date

```
from datetime import datetime
from crewai.tools import tool

@tool("Current Date")
def current_date(text: str):
    return datetime.now().strftime("%d-%m-%Y")
```

---

7. Mini Project

AI Company Research Assistant

Problem

A sales executive wants to contact a company.

Before sending an email they need:

- Company overview
- Products
- Industry
- AI opportunities

Instead of doing everything manually, multiple agents collaborate.

Workflow

```
Company Name
      |
      ▼
Research Agent
      |
      ▼
Business Analysis Agent
      |
      ▼
Email Writer Agent
      |
      ▼
Final Report
```

Step 1: Agents

```python
from crewai import Agent

research_agent = Agent(
    role="Research Analyst",
    goal="Research the company",
    backstory="Expert business researcher",
    verbose=True
)

analysis_agent = Agent(
    role="Business Consultant",
    goal="Find business opportunities",
    backstory="AI consultant",
    verbose=True
)

email_agent = Agent(
    role="Sales Executive",
    goal="Write a personalized email",
    backstory="Professional sales writer",
    verbose=True
)
```

Step 2: Tasks

```
from crewai import Task

research = Task(
    description="Research Microsoft company",
    expected_output="Detailed company report",
    agent=research_agent
)

analysis = Task(
    description="Suggest AI solutions for Microsoft",
    expected_output="Business recommendations",
    agent=analysis_agent
)

email = Task(
    description="Write a personalized cold email",
    expected_output="Professional email",
    agent=email_agent
)
```

Step 3: Crew

```
from crewai import Crew

crew = Crew(
    agents=[
        research_agent,
        analysis_agent,
        email_agent
    ],
    tasks=[
        research,
        analysis,
        email
    ],
    verbose=True
)

result = crew.kickoff()
```

```
print(result)
```

Expected Output

```
Company: Microsoft

Industry:
Technology

Products:
Azure
Microsoft 365
Windows
GitHub

AI Opportunities:
• Customer support automation
• Document intelligence
• Sales automation

Cold Email:

Dear Team,

After researching Microsoft, I identified several areas where AI-powered
automation can improve operational efficiency...

Best Regards,
Your Name
```

Real-World Applications

- AI Customer Support
- AI HR Recruitment
- AI Research Assistant
- AI Financial Analyst
- AI Legal Advisor
- AI Sales Automation
- AI Healthcare Assistant
- AI Business Consultant

Advantages of CrewAI

- Easy to learn
- Modular architecture
- Role-based collaboration
- Reusable agents
- Tool integration
- Supports complex workflows
- Suitable for enterprise AI systems

Practice Exercises

1. Build a multi-agent travel planner with Research, Budget, and Itinerary agents.
2. Create a student career advisor using Career, Skills, and Course Recommendation agents.
3. Develop an AI news summarizer with Web Search, Summarization, and Editor agents.
4. Create an AI resume reviewer with Resume Analysis, ATS Scoring, and Interview Preparation agents.
5. Extend the Company Research Assistant by adding:
6. A web search tool
7. A PDF report generator
8. An email sending tool
9. A CRM integration tool
10. A manager agent that delegates tasks automatically

These exercises help students understand how to design practical, production-style multi-agent systems using CrewAI.